# Trabajo Práctico Nº 3

## Computación en la Nube (Kubernetes / RabbitMQ)

**Fecha de Entrega: 21/04/26**

**(todos los puntos)**

---

## Requisitos, consideraciones y formato de entrega

- **Integrar herramientas de IA en su ciclo de vida de desarrollo** (Cursor, ChatGPT/Codex, Claude, GitHub Copilot, etc.). Se espera que las utilicen como asistentes para codificar, depurar y documentar. En el informe, mencionen qué herramientas usaron y cómo les ayudaron.
- **Se puede implementar con cualquier lenguaje dentro de los que se mencionaron en clase (node, python, java).**
- **Deben incluir una grabación video** que se debe subir al repositorio donde se expliquen los servicios, componentes y configuraciones que tomaron en cuenta. Esto debe mostrar que comprenden cada punto y su desarrollo.
- **Pruebas Unitarias y de Integración:** Incluir un conjunto mínimo de pruebas automatizadas que cubren las funcionalidades críticas del proyecto.
- Generar un **informe detallado** que incluya respuestas a consultas, métricas y tiempos de evaluación, gráficas, diagramas de arquitectura y conclusiones.
- Mantener un **repositorio público** en un servicio de control de versiones como GitHub, Bitbucket o GitLab. Cada ejercicio (HIT #) debe contar con una carpeta y un README.md explicativo.
    - El README.md de cada Hit debe incluir como mínimo: instrucciones para ejecutar el proyecto, diagrama de arquitectura, y decisiones de diseño tomadas.
- Compilar la aplicación para ejecución desde la terminal, con recursos preparados para ser desplegados directamente sin necesidad de abrir un IDE.
- Implementar un **pipeline de CI/CD** que automatice la compilación y el despliegue de la aplicación con cada nueva versión de código (GitHub Actions).
- Desplegar el servicio en un **entorno público** accesible desde Internet para su evaluación en producción (Despliegue en la nube).

- Proporcionar un **endpoint público** para cada servicio que permita verificar el estado de los principales servicios. No requiere GUI, puede devolver un JSON (Key=Servicio, value=Status). Ejemplos: https://status.lemon.me/, https://health.aws.amazon.com/health/status.
- Gestionar y mantener **registros de actividades (logs)** en memoria y disco.
- **Seguridad:**
    - No commitear .env, credenciales ni secrets al repositorio. Configurar .gitignore apropiado desde el inicio.
    - Gestionar credenciales por ambiente de forma segura (GitHub Secrets, Secret Manager / Parameter Store). **Zero static keys**: autenticarse contra cloud providers vía Workload Identity / OIDC.
    - Para registros Docker: usar image pull secrets o Workload Identity en lugar de enviar credenciales en payloads.
    - Incluir gitleaks en el pipeline de CI — si detecta un secret hardcodeado, el pipeline debe fallar.
    - Si expuso un secret accidentalmente, revocarlo de inmediato y generar uno nuevo (queda en el historial de Git aunque lo elimine).

---

## Contenidos del programa relacionados

- **U3.1** Cloud Computing: principios y aplicaciones.
- **U4.3** Contenedores Docker: imágenes, Dockerfile, docker-compose.
- **U4.4** Orquestación con Kubernetes: Pods, Services, Deployments, autoscaling, tolerancia a fallos.
- **U4.5** Características de la nube: elasticidad, tipos de nube (híbrida), Cloud-Bursting.
- **U5.1–U5.4** DevOps y CI/CD: pipelines de despliegue, construcción de imágenes Docker, despliegues sobre Kubernetes.
- **U5.5** Observabilidad: logging, métricas y monitoreo (Prometheus, Grafana).
- **U5.6** Infraestructura como Código (IaC): Terraform.
- **U7.5** Esquemas algorítmicos paralelos: Maestro/Esclavo, Granja de Trabajadores.

---

# Práctica

En la nube, tengo 2 patrones de trabajo:

- Cloud Native (corre 100% en la nube).
- **Híbrido (parte en equipos locales / On-premise - Cloud)**

Ahora, pensemos la nube como una extensión de la capacidad de cómputo de su/sus equipo/s. De esta manera, logramos aplicar el patrón de **Cloud-Bursting** (ver Ref. 10).

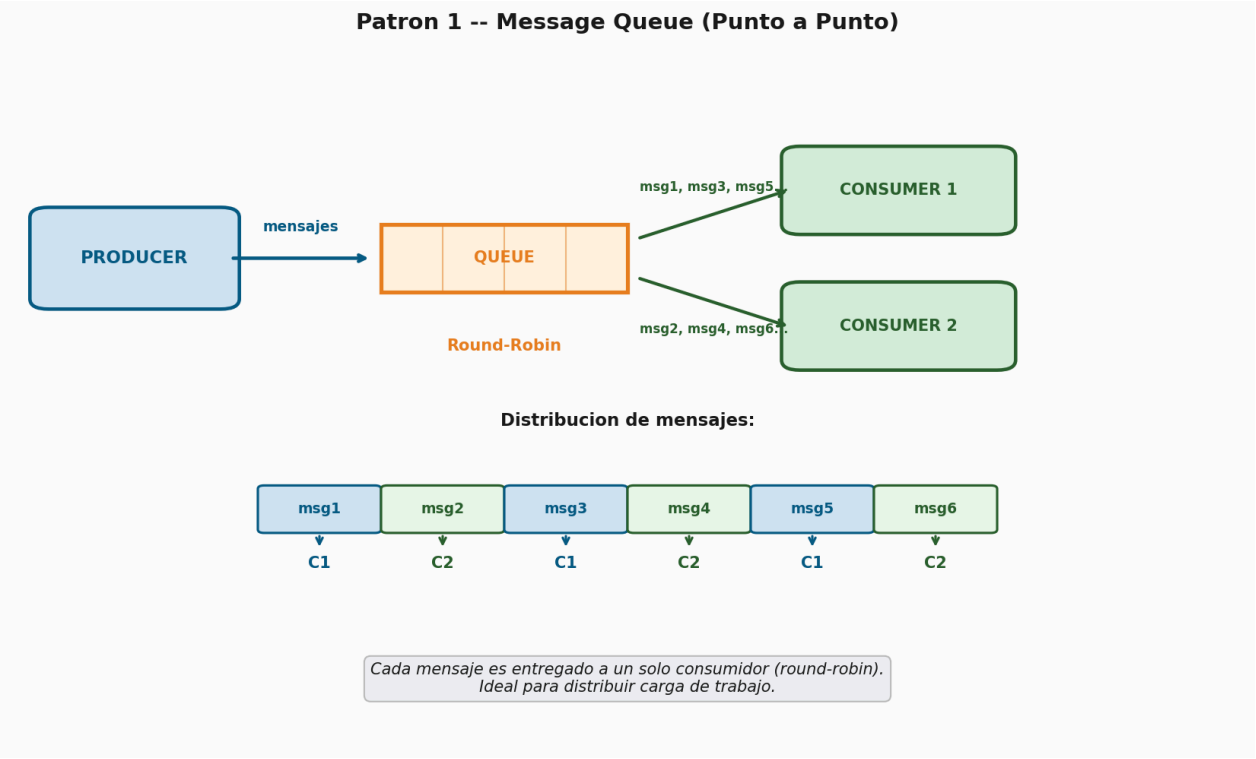
Por ejemplo, si tenemos una aplicación Cliente/Servidor (HTTP, gRPC, WSS) como por ejemplo filtro Sobel…

---

## Hit #0 - Patrones de Mensajería con RabbitMQ

Antes de comenzar con el procesamiento distribuido de imágenes, es fundamental comprender los patrones de mensajería que utilizará a lo largo de este TP y del TP Integrador. RabbitMQ será su broker de mensajes, y es necesario que entienda cómo funciona y qué patrones soporta.

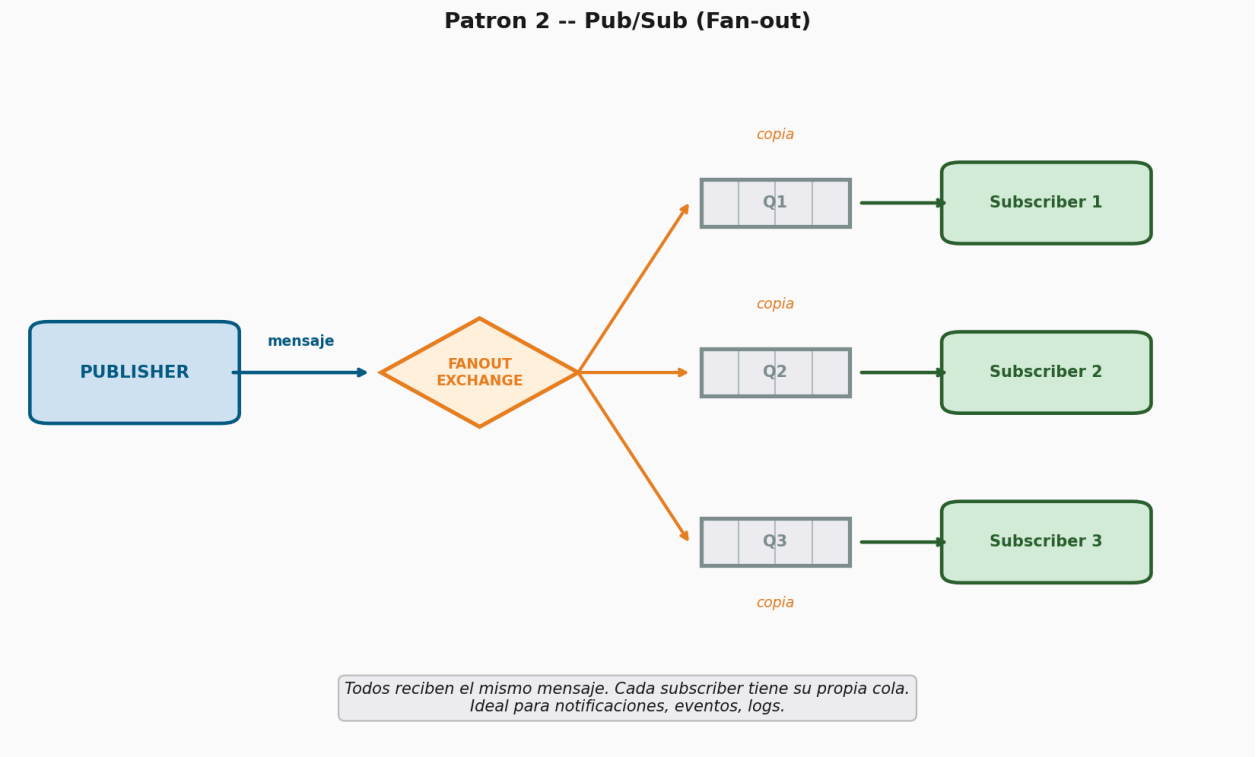
Implemente los siguientes 4 ejemplos simples utilizando RabbitMQ (puede usar Docker para levantar el broker: docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:management). Cada ejemplo debe ser un programa funcional independiente con su código en una subcarpeta:



Patrón 1 - Message Queue

### *Ejemplo 1 - Message Queue (punto a punto)*

Un productor envía mensajes a una cola y **UN solo consumidor** los recibe. Cada mensaje es procesado exactamente por un consumidor. Implemente un productor que envíe 10 tareas numeradas y un consumidor que las reciba y las imprima. Luego levante 2 consumidores y observe cómo RabbitMQ distribuye los mensajes entre ambos (round-robin). Documente el comportamiento observado.

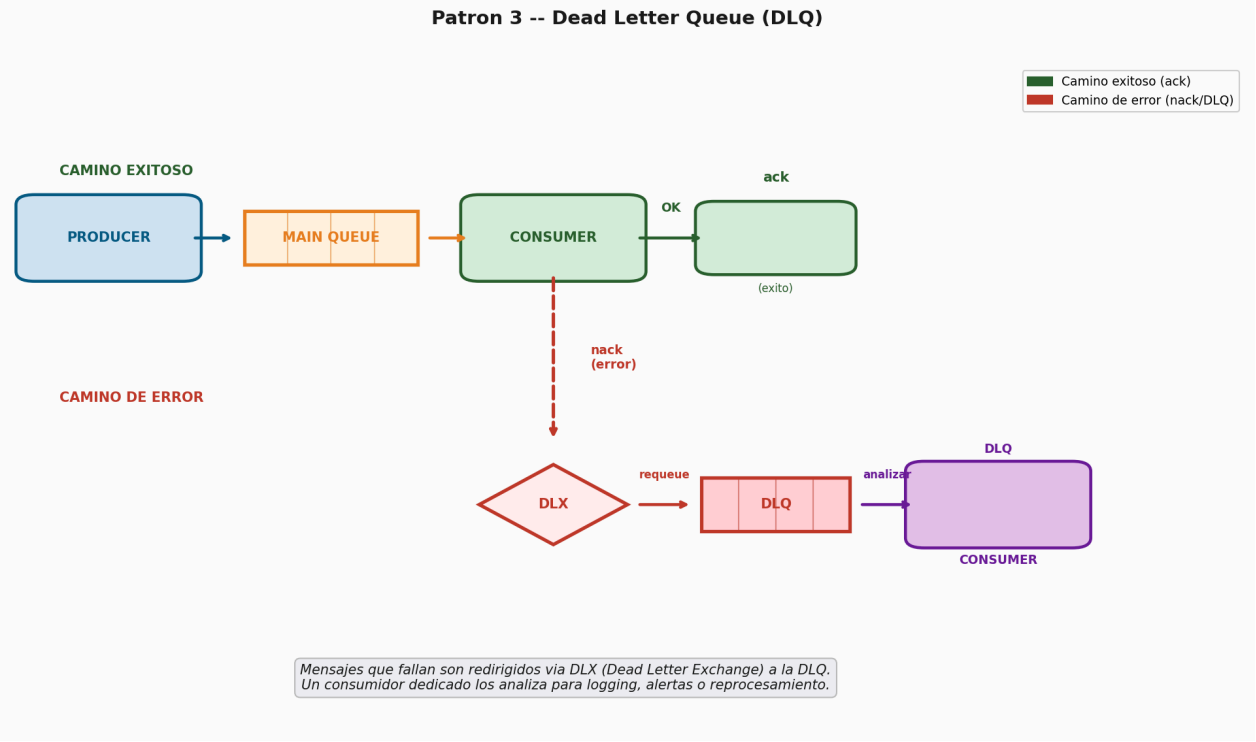



Patrón 2 - Pub/Sub Fan-out

### *Ejemplo 2 - Event Bus / Pub-Sub (fan-out)*

Un productor publica eventos en un exchange de tipo fanout y **TODOS los consumidores** suscritos reciben una copia del mensaje. Implemente un publicador que emita eventos de “nuevo bloque minado” y 3 suscriptores que representen nodos de la red que deben recibir la notificación. Verifique que los 3 reciben el mismo mensaje.

En la industria, este patrón lo implementan servicios como AWS EventBridge, Google Pub/Sub y Apache Kafka.

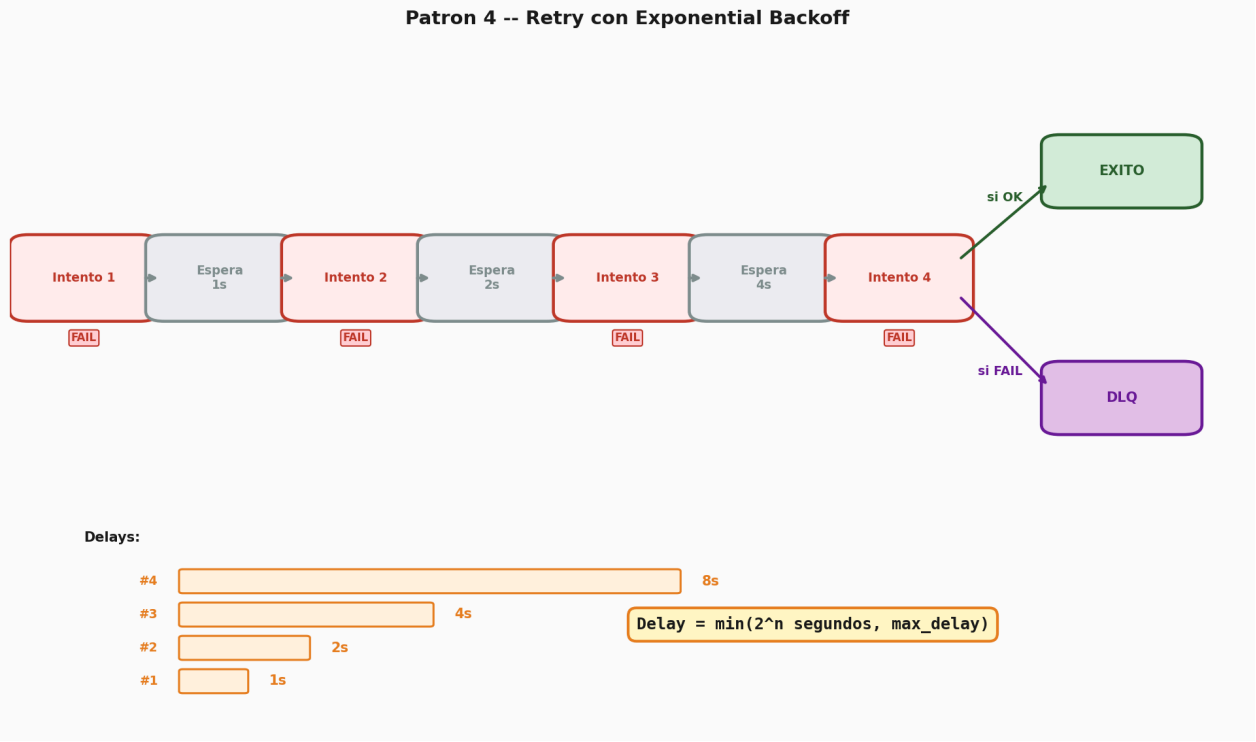



Patrón 3 - Dead Letter Queue

## *Ejemplo 3 - Dead Letter Queue (DLQ)*

Configure una cola principal con una **Dead Letter Exchange (DLX)** en RabbitMQ. El consumidor debe rechazar (nack) los mensajes que contengan un campo "error": true, y estos deben ser redirigidos automáticamente a la cola de Dead Letters. Implemente un segundo consumidor que lea la DLQ e imprima los mensajes fallidos.

Este patrón es fundamental para no perder mensajes que no pudieron ser procesados. En la industria se usa en SQS + DLQ (AWS) y RabbitMQ DLX.




Patrón 4 - Retry con Exponential Backoff

### *Ejemplo 4 - Retry con Exponential Backoff*

Implemente un consumidor que al recibir un mensaje intente procesarlo, y si falla (simule un fallo aleatorio con probabilidad del 50%), lo reencole con un delay creciente: 1s, 2s, 4s, 8s (exponential backoff). Después de 4 reintentos fallidos, envíe el mensaje a la DLQ del ejemplo anterior.

Para implementar el delay puede usar el plugin rabbitmq_delayed_message_exchange o TTL por mensaje con una cola intermedia. Registre en un log el número de intento y el tiempo de espera de cada reintento.

Este patrón es estándar en la industria para manejar fallos transitorios (servicios caídos, timeouts, rate limits).

*Documente en su informe: un diagrama de arquitectura de cada patrón, las diferencias entre ellos, y en qué escenarios usaría cada uno. Esta base le servirá para los hits siguientes donde aplicará estos patrones al procesamiento distribuido de imágenes.*

---

## Hit #1 - El operador de Sobel

El operador de Sobel es una máscara que, aplicada a una imagen, permite detectar (resaltar) bordes. Este operador es una operación matemática que, aplicada a cada pixel y teniendo en

cuenta los píxeles que lo rodean, obtiene un nuevo valor (color) para ese pixel. Aplicando la operación a cada píxel, se obtiene una nueva imagen que resalta los bordes.

**Objetivo:**

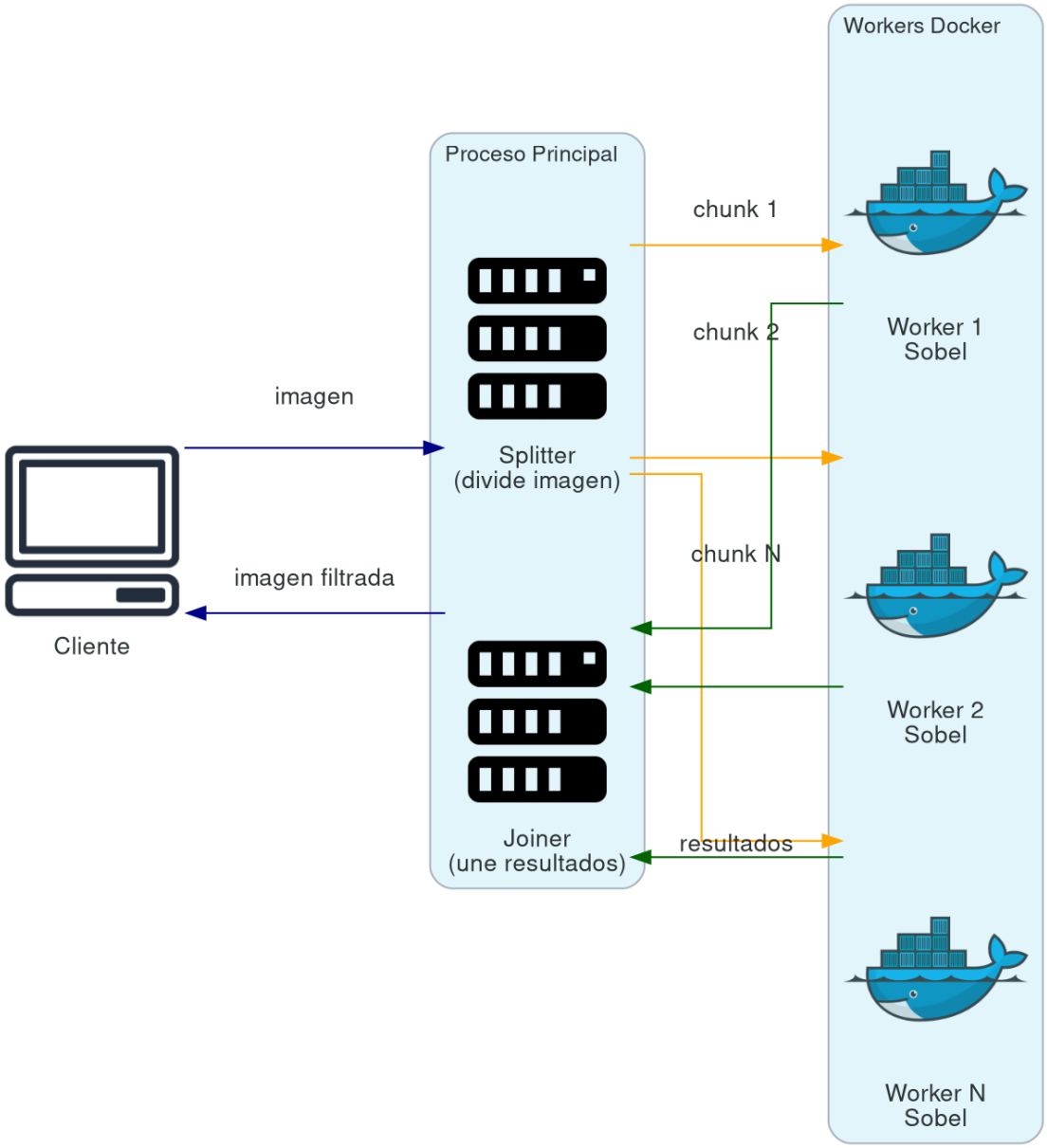
- Input: una imagen.
- Proceso: Sobel.
- Output: una imagen filtrada.

**Parte 1:** Desarrollar un proceso centralizado que tome una imagen, aplique la máscara, y genere un nuevo archivo con el resultado.

**Parte 2:** Desarrolle este proceso **de manera distribuida** donde se debe partir la imagen en N pedazos, y asignar la tarea de aplicar la máscara a N procesos distribuidos. Después deberá unificar los resultados.

A partir de ambas implementaciones, comente los resultados de performance dependiendo de la cantidad de nodos y **tamaño de imagen**.

**Parte 3:** Mejore la aplicación del punto anterior para que, en caso de que un proceso distribuido (al que se le asignó parte de la imagen a procesar - **WORKER**) se caiga y no responda, el proceso principal detecte esta situación y pida este cálculo a otro proceso.




Hit #1 -- Sobel Distribuido con Workers Docker

Hit #1 - Sobel Distribuido con Workers Docker

---

Bien, todo eso lo corrieron “distribuido” pero “centralizado” en su propia computadora. Sin embargo, la necesidad de procesamiento para aplicar un filtro sobre imágenes (que pueden ser muy grandes) puede generar que nos encontremos con un tope de capacidad de cómputo (más allá que lo hayamos construido de forma redundante). ¿Qué opciones tenemos?

- Si es local, agregar más nodos de trabajo. Sin embargo, como discutimos, eso requiere tiempo (compra, ambientación, espacio e instalación de paquetes) y produce un Costo Fijo mínimo (más allá del variable por intensidad de uso).
- Hacer un offloading a la nube (ver Ref. 11). La cual trabajaremos en este apartado.

---

## Hit #2 - Sobel con offloading en la nube

Para construir una base elástica:

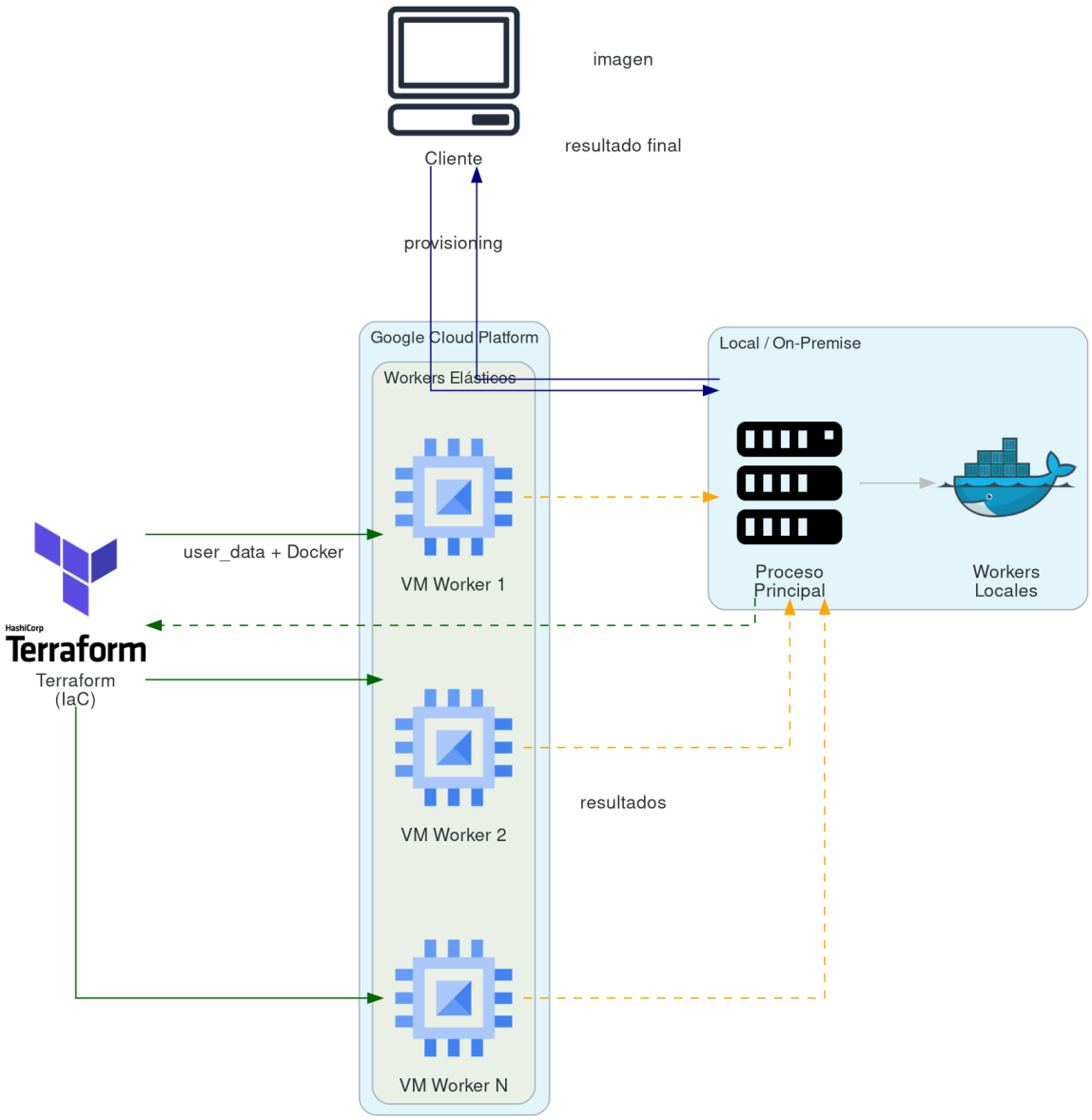
Mismo objetivo de calcular Sobel, pero ahora vamos a usar **Terraform** para construir nodos de trabajo cuando se requiera procesar tareas y eliminarlos al terminar. Recuerde que será necesario:

- Instalar con #user_data las herramientas necesarias (java, docker, tools).
- Copiar ejecutable (jar, py, etc) o descargar imagen Docker (hub).
- Poner a correr la aplicación e integrarse al cluster de trabajo.

### *Requisitos de Infraestructura como Código (IaC)*

- **Remote state obligatorio:** Configure un backend remoto para el estado de Terraform (GCS bucket + lock o S3 + DynamoDB lock). No se acepta estado local (terraform.tfstate en el repo).
- **Estructura de archivos mínima:** Organice su código Terraform en archivos separados: provider.tf (configuración del provider y backend), variables.tf (declaración de variables), main.tf (recursos principales), outputs.tf (valores de salida). No coloque todo en un único archivo.
- **Terraform plan en CI/CD:** Su pipeline debe ejecutar terraform plan en cada Pull Request y terraform apply solo al mergear a la rama principal.

El objetivo de este ejercicio es que ustedes puedan construir una arquitectura escalable (tipo 1, inicial) **HÍBRIDA**. Debe presentar el diagrama de arquitectura y comentar su decisión de desarrollar cada servicio y dónde lo “coloca”.




Hit #2 -- Sobel con Offloading en la Nube (Híbrido)

Hit #2 - Sobel con Offloading en la Nube

---

## Hit #3 - Sobel contenerizado asincrónico y escalable (BASE DE TP FINAL)

A diferencia del clúster anterior, la idea es que construya una infraestructura basada en la nube pero ahora con un enfoque diferente.

Para ello, será necesario:

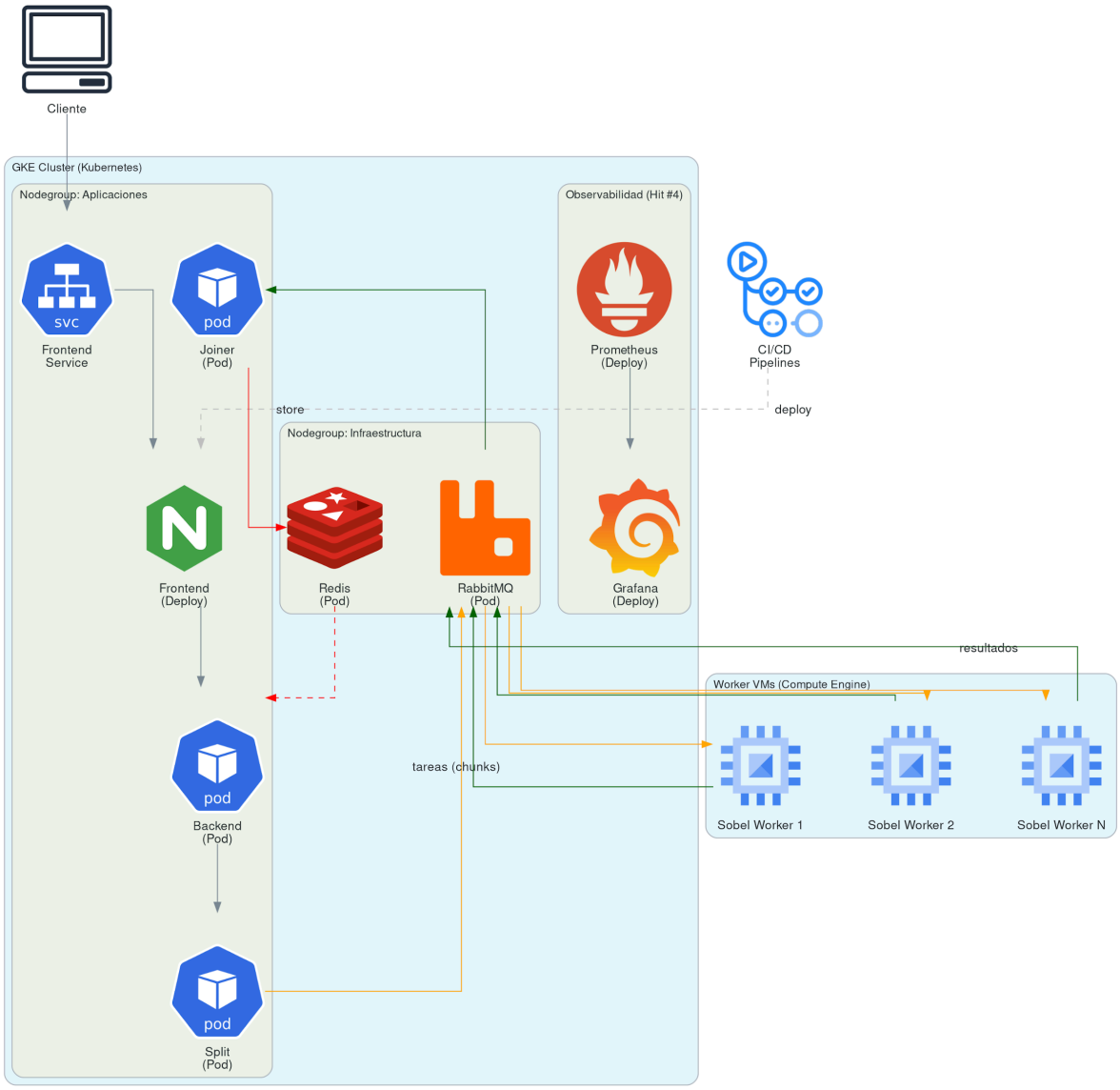
### *1. Desplegar con Terraform un cluster de Kubernetes (GKE)*

Este será el manejador de todos los recursos que vayamos a desplegar. Es decir, va a alojar tanto los servicios de infraestructura (RabbitMQ y Redis) como los componentes de las aplicaciones que vamos a correr (frontend, backend, split, joiner, etc). Este clúster tiene que tener la siguiente configuración mínima:

1. Un nodegroup para alojar los servicios de infraestructura (RabbitMQ, Redis, otros).
2. Un nodegroup compartido para las aplicaciones del sistema (front, back, split, joiner).
3. Máquinas virtuales (fuera del cluster) que se encarguen de las tareas de procesamiento / cómputo intensivo.

### *Requisitos de mensajería (aplicando los patrones del Hit #0)*

- **Dead Letter Queue:** Configure una DLX en las colas de tareas de procesamiento Sobel. Si un worker falla al procesar un fragmento de imagen (crash, timeout, out of memory), el mensaje debe ir a la DLQ en lugar de perderse. Implemente un servicio que monitoree la DLQ y re-asigne los fragmentos fallidos a otros workers.
- **Retry con exponential backoff:** Cuando un worker no puede contactar a RabbitMQ o Redis (por ejemplo, durante un despliegue o reinicio), debe reintentar la conexión con backoff exponencial (1s, 2s, 4s, 8s, máximo 30s). No implementar un retry infinito sin backoff: esto genera tormentas de reconexión que empeoran el problema.
- **Pub/Sub para notificación de resultados:** Cuando un worker completa un fragmento, publique el resultado en un exchange fanout para que tanto el servicio joiner como el dashboard de monitoreo reciban la notificación.




Hit #3 -- Arquitectura Kubernetes (GKE) con Sobel Distribuido

Hit #3 - Arquitectura Kubernetes (GKE)

## *2. Construir los pipelines de despliegue de todos los servicios*

- **Pipeline 1:** El que construye el Kubernetes.
    - Pipeline 1.1: El que despliega los servicios (base datos - Redis, sistema de colas - RabbitMQ).
    - Pipeline 1.2-1.N: De cada aplicación desarrollada (frontend, backend, split, join).
- **Pipeline 2:** Despliegue de máquinas virtuales para construir a los workers. Objetivo deseable: que estas máquinas sean “dinámicas”.

---

## Hit #3 (cont.) - Análisis de Desempeño Bajo Carga

Para evaluar el desempeño de la plataforma, se analizarán los tiempos de respuesta en diferentes escenarios de carga, modificando las siguientes variables:

- **Tamaño de los datos (V1):** Se utilizarán tamaños representativos de carga, tales como 1 KB, 10 KB, 1 MB, 10 MB y 100 MB.
- **Cantidad de peticiones concurrentes (V2):** Se simularán diferentes niveles de concurrencia.
- **Cantidad de workers (V3):** Se ajustará el número de procesos o threads disponibles para manejar las peticiones.

Disclaimer: Si quiere lograr usar más nodos, use nodos en distintas regiones.

El objetivo es comprender cómo la plataforma responde al modificar estas variables, permitiendo identificar su capacidad de escalabilidad y posibles puntos de mejora. Los resultados se presentarán en una tabla que muestre los tiempos de respuesta obtenidos en cada combinación de las variables, proporcionando una visión clara de la evolución del desempeño bajo diferentes condiciones de carga.

**Herramienta de Load Testing:** Para las pruebas de carga, utilice una herramienta de benchmarking como Locust o k6. Estas herramientas permiten definir escenarios de carga reproducibles y generar reportes comparables. Documente la configuración del test (cantidad de usuarios virtuales, ramp-up, duración) y presente los resultados con métricas estándar: p50, p95, p99 de latencia, throughput (req/s) y tasa de errores.

---

## Hit #4 - Observabilidad (Prometheus + Grafana)

Despliegue Prometheus y Grafana en el cluster de Kubernetes para monitorear la plataforma. Para ello:

1. **Instalación:** Instale Prometheus y Grafana en el nodegroup de infraestructura (puede utilizar Helm charts oficiales: prometheus-community/kube-prometheus-stack).
2. **Instrumentación:** Instrumente sus servicios (backend, workers, split, joiner) para que exporten métricas custom: tareas procesadas, tareas en cola, tiempo de procesamiento por tarea, y errores.
3. **Dashboard:** Cree un dashboard en Grafana que muestre al menos: uso de CPU y memoria por pod/nodo, mensajes procesados en RabbitMQ (publicados vs consumidos vs en cola), latencia de procesamiento de tareas (p50, p95, p99), y tasa de errores.
4. **Alertas:** Configure al menos una alerta básica (por ejemplo: si la cola de RabbitMQ supera un umbral o si un worker no responde en X segundos).

## Referencias y Bibliografía


- **[BUR18]** Burns, B. (2018). *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services*. O'Reilly Media.
- **[CBURST]** Cloud-bursting — Atlassian. https://www.atlassian.com/es/continuous-delivery/principles/cloud-bursting
- **[GITLEAKS]** gitleaks — Protect and discover secrets using Gitleaks. https://github.com/gitleaks/gitleaks
- **[GRAF]** Grafana Documentation. https://grafana.com/docs/grafana/
- **[K6]** k6 — Load testing for engineering teams. https://k6.io
- **[K8S]** Kubernetes Documentation. https://kubernetes.io/docs/
- **[KUM10]** Kumar, K. & Lu, Y. (2010). "Cloud Computing for Mobile Users: Can Offloading Computation Save Energy?". Purdue University. https://www.cs.purdue.edu/homes/bb/mobile-cloud-survey.pdf
- **[LOCUST]** Locust — Open source load testing tool. https://locust.io
- **[PROM]** Prometheus — Monitoring system & time series database. https://prometheus.io/docs/
- **[RMQ]** RabbitMQ Documentation — Tutorials y patrones de mensajería. https://www.rabbitmq.com/tutorials
- **[SOB68]** Sobel, I. & Feldman, G. (1968). "A 3x3 Isotropic Gradient Operator for Image Processing". Stanford AI Project.
- **[TF]** Terraform by HashiCorp — Documentation. https://developer.hashicorp.com/terraform/docs